🏏 IPL Win Probability Predictor

Data Cleaning, Feature Engineering & EDA

📋 What we'll do today

Step	What
1	Import libraries
2	Load & explore the dataset
3	Clean the data
4	Engineer new features
5	Exploratory Data Analysis (EDA)
6	Save cleaned data for Day 2

1️⃣ Import Libraries

Library	What it does
`pandas`	Reads CSVs, creates tables (DataFrames), filters and transforms data
`numpy`	Math operations on large arrays of numbers
`matplotlib`	Creates charts and graphs
`warnings`	Suppresses unnecessary warning messages so output stays clean

```
# pandas — the main library for working with data tables (DataFrames)
import pandas as pd

# numpy — for numerical operations and handling infinity/NaN values
import numpy as np

# matplotlib — for plotting charts and graphs
import matplotlib.pyplot as plt

# warnings — hide unnecessary output so our notebook stays clean
import warnings
warnings.filterwarnings('ignore')  # 'ignore' means don't show warnings
```

2️⃣ Load & Explore the Dataset

Our dataset is a single CSV file with **ball-by-ball IPL data from 2008–2025**. Each row = one ball bowled in a match.

Key columns we care about

Column	What it means
`innings`	1 = first team batting, 2 = second team batting (chasing)
`batting_team`	Team currently batting
`bowling_team`	Team currently bowling
`city`	City where the match is played
`team_runs`	Total runs scored so far this innings
`team_balls`	Total valid balls faced so far
`team_wicket`	Total wickets fallen so far
`runs_target`	Target score (only in 2nd innings)
`match_won_by`	Name of the team that won the match

```
# pd.read_csv() reads the CSV file and converts it into a DataFrame
# A DataFrame is like an Excel table inside Python
df = pd.read_csv('/content/IPL.csv')

# .shape tells us (number of rows, number of columns)
print('Shape:', df.shape)
```

```
# .head() shows the first 5 rows so we can see what the data looks like
df.head()
```

```
Shape: (121151, 64)
```

	Unnamed: 0	match_id	date	match_type	event_name	innings	batting_team	bowling_team	over	ball	...	team_runs	team_ba
0	131970	335982	2008-04-18	T20	Indian Premier League	1	Kolkata Knight Riders	Royal Challengers Bangalore	0	1	...	1	
1	131971	335982	2008-04-18	T20	Indian Premier League	1	Kolkata Knight Riders	Royal Challengers Bangalore	0	2	...	1	
2	131972	335982	2008-04-18	T20	Indian Premier League	1	Kolkata Knight Riders	Royal Challengers Bangalore	0	3	...	2	
3	131973	335982	2008-04-18	T20	Indian Premier League	1	Kolkata Knight Riders	Royal Challengers Bangalore	0	3	...	2	
4	131974	335982	2008-04-18	T20	Indian Premier League	1	Kolkata Knight Riders	Royal Challengers Bangalore	0	4	...	2	

5 rows × 64 columns

```
# .isnull() checks every cell — True if missing, False if not
# .sum() counts how many True values (missing) are in each column
df.isnull().sum()
```

	0
Unnamed: 0	0
match_id	0
date	0
match_type	0
event_name	0
...	...
batter_balls	1
bowler_wicket	1
batting_partners	1
next_batter	115378
striker_out	1

64 rows × 1 columns

dtype: int64

3️⃣ Data Cleaning

Raw data is never perfect. We need to:

1. **Filter** — keep only 2nd innings rows (chasing team)
2. **Drop** — remove rows with missing target or unknown city
3. **Rename** — standardise old franchise names to current ones
4. **Filter** — keep only the current 10 active IPL teams

Why only 2nd innings?

We're predicting **win probability for the chasing team**.

- 1st innings: no target yet, nothing to chase — prediction doesn't make sense

- 2nd innings: team is chasing a target — this is where our model applies

```
# Keep only rows where innings == 2 (the chasing team's deliveries)
# .copy() creates an independent copy so we don't accidentally modify the original
df = df[df['innings'] == 2].copy()

# dropna(subset=[...]) removes rows where the specified column is missing
# runs_target is NaN in 1st innings rows — we've already removed those,
# but this is a safety check
df = df.dropna(subset=['runs_target'])

# Remove rows where city is 'Unknown' — we can't use unknown cities
# as a feature for the model
df = df[df['city'] != 'Unknown']

# Print how many rows remain after cleaning
print('Rows remaining:', len(df))
```

```
Rows remaining: 56920
```

```
# Dictionary mapping old names → new names
team_rename = {
    'Delhi Daredevils'            : 'Delhi Capitals',        # renamed in 2019
    'Deccan Chargers'             : 'Sunrisers Hyderabad',   # replaced in 2013
    'Kings XI Punjab'             : 'Punjab Kings',           # renamed in 2021
    'Rising Pune Supergiant'      : 'Rising Pune Supergiants', # name fix
    'Royal Challengers Bangalore' : 'Royal Challengers Bengaluru', # renamed in 2023
}

# Apply the rename to all three team-related columns
# .replace() swaps old values with new values based on the dictionary
for col in ['batting_team', 'bowling_team', 'match_won_by']:
    df[col] = df[col].replace(team_rename)
```

```
# List of current active IPL franchises (10 teams)
TEAMS = ['Chennai Super Kings', 'Delhi Capitals', 'Gujarat Titans',
         'Kolkata Knight Riders', 'Lucknow Super Giants', 'Mumbai Indians',
         'Punjab Kings', 'Rajasthan Royals', 'Royal Challengers Bengaluru',
         'Sunrisers Hyderabad']

# .isin(TEAMS) returns True only if the value is in our TEAMS list
# We apply this to BOTH batting and bowling team columns using &  (AND)
# This removes defunct teams like Kochi Tuskers, Pune Warriors etc.
df = df[df['batting_team'].isin(TEAMS) & df['bowling_team'].isin(TEAMS)]

# Verify — should show exactly 10 teams
print('Teams:', sorted(df['batting_team'].unique()))
```

```
Teams: ['Chennai Super Kings', 'Delhi Capitals', 'Kolkata Knight Riders', 'Mumbai Indians', 'Punjab Kings', 'Rajasthan Roya
```

4️⃣ Feature Engineering

Feature Engineering = creating new columns from existing ones that are more useful for the model.

The raw data tells us what happened ball by ball. But the model needs to understand **the current match situation** — things like:

- How many runs are still needed?
- How many balls are left?
- Is the batting team scoring fast enough?

Features we'll create

Feature	Formula	What it tells the model
`runs_left`	target – runs scored	How many more runs needed
`balls_left`	120 – balls faced	How many balls remaining (T20 = 120 balls)
`wickets_remaining`	10 – wickets fallen	How many wickets still in hand
`crr`	runs × 6 / balls	Current run rate (runs per over)

Feature	Formula	What it tells the model
`rrr`	runs_left × 6 / balls_left	Required run rate (runs per over needed)
`result`	batting_team == match_won_by	**Target**: 1 if batting team wins, 0 if loses

> **Why multiply by 6?** Cricket uses overs (6 balls each). Dividing balls by 6 converts to overs.

```
# Remove rows where team_balls = 0 (the very first ball of the innings)
# We can't divide by 0 when calculating run rates
df = df[df['team_balls'] > 0]

# How many more runs does the batting team need to win?
df['runs_left'] = df['runs_target'] - df['team_runs']

# How many valid balls are remaining? (T20 = 120 balls total)
# team_balls counts only legal deliveries (not wides/no-balls)
df['balls_left'] = 120 - df['team_balls']

# How many wickets does the batting team still have?
df['wickets_remaining'] = 10 - df['team_wicket']

# Current Run Rate = runs scored so far per over
# (team_balls / 6) converts balls to overs
df['crr'] = (df['team_runs'] * 6 / df['team_balls']).round(2)

# Required Run Rate = runs still needed per over remaining
df['rrr'] = (df['runs_left'] * 6 / df['balls_left']).round(2)

# Target variable: 1 if the batting team won, 0 if they lost
# This is what the model will learn to predict
df['result'] = (df['batting_team'] == df['match_won_by']).astype(int)
```

```
# Select only the columns we need — 9 features + 1 target
FEATURES = ['batting_team', 'bowling_team', 'city',
            'runs_left', 'balls_left', 'wickets_remaining',
            'runs_target', 'crr', 'rrr', 'result']

# Create a clean copy with only these columns
final_df = df[FEATURES].copy()

# Replace infinity values (happen when dividing by zero) with NaN
# Then drop any rows that have NaN in any column
# np.inf = positive infinity, -np.inf = negative infinity
final_df = final_df.replace([np.inf, -np.inf], np.nan).dropna()

# Confirm the final shape of our clean dataset
print('Final shape:', final_df.shape)
final_df.head()
```

```
Final shape: (50050, 10)
```

	batting_team	bowling_team	city	runs_left	balls_left	wickets_remaining	runs_target	crr	rrr	result
124	Royal Challengers Bengaluru	Kolkata Knight Riders	Bangalore	222.0	119	10	223.0	6.0	11.19	0
125	Royal Challengers Bengaluru	Kolkata Knight Riders	Bangalore	221.0	119	10	223.0	12.0	11.14	0
126	Royal Challengers Bengaluru	Kolkata Knight Riders	Bangalore	221.0	118	10	223.0	6.0	11.24	0
127	Royal Challengers Bengaluru	Kolkata Knight Riders	Bangalore	220.0	117	10	223.0	6.0	11.28	0

Next steps: Generate code with `final_df` New interactive sheet

5 Exploratory Data Analysis (EDA)

EDA = exploring the data visually to understand patterns before training the model.

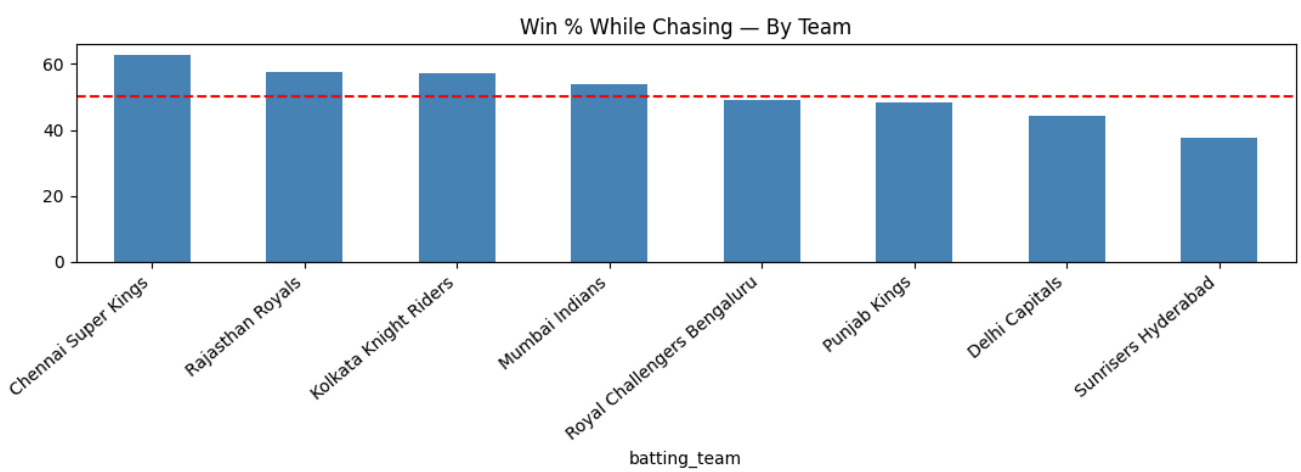
We want to answer questions like:

- Which teams win more often when chasing?
- What does the run rate distribution look like?

```
# groupby('batting_team') — splits the DataFrame into groups, one per team
# ['result'] — we look at the result column for each group
# .mean() — average of result (since result is 0 or 1, mean = win percentage)
# * 100 — convert to percentage
# .round(1) — round to 1 decimal place
# .sort_values(ascending=False) — show highest win % at the top
win_pct = (final_df.groupby('batting_team')['result']
           .mean().sort_values(ascending=False) * 100).round(1)

print(win_pct)
```

```
batting_team
Chennai Super Kings            62.8
Rajasthan Royals               57.7
Kolkata Knight Riders          57.3
Mumbai Indians                 53.9
Royal Challengers Bengaluru    48.9
Punjab Kings                   48.3
Delhi Capitals                 44.3
Sunrisers Hyderabad            37.7
Name: result, dtype: float64
```

```
# .plot(kind='bar') creates a bar chart from the Series
win_pct.plot(kind='bar', figsize=(11, 4), color='steelblue')

# Add a title to the chart
plt.title('Win % While Chasing — By Team')

# Draw a horizontal red dashed line at 50% for reference
plt.axhline(50, color='red', linestyle='--', label='50%')

# Rotate x-axis labels so team names don't overlap
plt.xticks(rotation=40, ha='right')

# Automatically adjust spacing so nothing gets cut off
plt.tight_layout()

# Display the chart
plt.show()
```



6️⃣ Save Cleaned Dataset

We save the cleaned dataset so we don't have to redo all the cleaning on Day 2. On Day 2 we'll load this file directly and jump straight into Machine Learning.

> **Colab users:** After running this cell, right-click `IPL_processed_dataset.csv` in the Files panel → Download. Save it to your computer.

```
# .to_csv() saves the DataFrame as a CSV file
# index=False means don't write the row numbers (0,1,2...) as a column
final_df.to_csv('IPL_processed_dataset.csv', index=False)

print('Saved processed_dataset.csv')
print('Rows:', len(final_df))
print()
print('✅ Day 1 complete! Download processed_dataset.csv before closing.')
```

```
Saved processed_dataset.csv
Rows: 50050

✅ Day 1 complete! Download processed_dataset.csv before closing.
```